

AutoDRIVE Ecosystem: Particle Filter & Autonomous Racing Guide

1. Setup & Environments

Terminal 1: Devkit/API + Bridge

Bash

```
xhost local:root
sudo docker run --name autodrive_roboracer_api --rm -it --entrypoint
/bin/bash \
  --network=host --ipc=host \
  -v /tmp/.X11-unix:/tmp/.X11-unix:rw \
  -v /home/abdo-yasser/colcon_ws/src:/home/autodrive_devkit/src \
  --env DISPLAY=$DISPLAY --privileged \
  autodriveecosystem/autodrive_roboracer_api:2026-icra-compete

# Inside Container:
ros2 launch autodrive_roboracer bringup_graphics.launch.py
```

Terminal 2: Simulator

Bash

```
xhost local:root
docker run --name autodrive_roboracer_sim --rm -it --entrypoint
/bin/bash \
  --network=host --ipc=host \
  -v /tmp/.X11-unix:/tmp/.X11-unix:rw \
  --env DISPLAY \
  --device /dev/dri:/dev/dri \
  --privileged \
  autodriveecosystem/autodrive_roboracer_sim:2026-icra-compete

# Inside Container:
export LIBGL_ALWAYS_SOFTWARE=1
export MESA_GL_VERSION_OVERRIDE=4.5
export MESA_GLSL_VERSION_OVERRIDE=450
./AutoDRIVE\ Simulator.x86_64 -batchmode -nographics -ip 127.0.0.1 -
port 4567
```

2. Dependencies & Compilation (First Time Only)

Terminal 3: Build & Config

Bash

```
docker exec -it autodrive_roboracer_api bash

# Install Core Tools
apt update && apt install ros-humble-navigation2 ros-humble-nav2-
bringup python3-pip -y
pip3 install Cython

# Compile RangeLibc (C++ Math Library)
cd /home/autodrive_devkit/src/range_libc/pywrapper
python3 setup.py install
```

3. Configuration & Bug Fixes (The Integration Phase)

أ. إعدادات `rf2o_laser_odometry`

افتح ملف `rf2o_laser_odometry.launch.py` ، وتأكد أن الإعدادات كالتالي (مهم جداً إغلاق الـ TF لتجنب التضارب):

Python

```
parameters=[{
    'laser_scan_topic' :
'/autodrive/roboracer_1/lidar',
    'odom_topic' : '/odom_rf2o',
    'publish_tf' : False, # ⬅️ (CRITICAL FIX) لمنع
مع المحاكى TF تضارب الـ
    'base_frame_id' : 'roboracer_1',
    'odom_frame_id' : 'odom',
    'init_pose_from_topic' : '',
    'freq' : 10.0}],
```

ب. إعدادات (YAML & Python) `particle_filter`

في نفس التيرمينال، قم بتنفيذ الأوامر التالية لتوجيه الفلتر للقنوات الصحيحة:

Bash

```
# Auto-Fix Configuration Files (Topic Names & Methods)
sed -i 's/rmgpu/cddt/g'
/home/autodrive_devkit/src/particle_filter/config/localize.yaml
sed -i "s|scan_topic: .*|scan_topic: '/autodrive/roboracer_1/lidar'|g"
/home/autodrive_devkit/src/particle_filter/config/localize.yaml
sed -i "s|odometry_topic: .*|odometry_topic: '/odom_rf2o'|g"
/home/autodrive_devkit/src/particle_filter/config/localize.yaml
```

الإصلاح البرمجي الجذري في ملف البايثون (particle_filter.py):

1. إصلاح شجرة الـ TF والـ Fake Scan: يجب حذف علامة / من أسماء الـ Frames وتغيير اسم الليدار ليتطابق مع السيارة.

Bash

```
sed -i "s|'/map'|'map'|g"
/home/autodrive_devkit/src/particle_filter/particle_filter/particle_filter.py
sed -i "s|'/laser'|'roboracer_1'|g"
/home/autodrive_devkit/src/particle_filter/particle_filter/particle_filter.py
```

2. إصلاح المحاور المعكوسة (Axis Flip Fix):

في ملف particle_filter.py، ابحث عن دالة odomCB (سطر 351 تقريباً) واستبدل مصفوفة الـ position لتصبح هكذا:

Python

```
position = np.array([
    -msg.pose.pose.position.y,
    msg.pose.pose.position.x])
```

4. Final Build & Execution

بعد حفظ جميع التعديلات، نقوم ببناء الحزمتين معاً:

Bash

```
cd /home/autodrive_devkit
colcon build --packages-select rf2o_laser_odometry particle_filter
source install/setup.bash
```

تسلسل التشغيل الميداني (يجب فتح تيرمينال منفصل لكل أمر):

1. **خريطة العالم (TF):

```
ros2 run tf2_ros static_transform_publisher 0 0 0 0 0 0 map world
```

1. **

2. حساب الحركة (rf2o):

```
ros2 launch rf2o_laser_odometry rf2o_laser_odometry.launch.py
```

2. الفلتر والتموضع (PF):

```
ros2 launch particle_filter localize_launch.py
```

,

3. القيادة اليدوية (Teleop):

```
ros2 run autodrive_roboracer teleop_keyboard
```

5. RViz Configuration (Visual Output)

لإظهار البيانات بشكل صحيح، اتبع الإعدادات التالية في واجهة RViz:

العنصر	الإعدادات المطلوبة	القيمة / المسار
Global Options	Fixed Frame	map (اكتبها يدوياً ودوس Enter)

العنصر	الإعدادات المطلوبة	القيمة / المسار
Map	Topic & Durability	/map & Transient Local
PoseArray	Topic	/pf/viz/particles
Pose	Topic	/pf/viz/inferred_pose

خطوات الاستلام على الخريطة:

1. استخدم أداة **2D Pose Estimate** في RViz. اضغط على موقع السيارة واسحب الماوس في نفس اتجاه مقدمة السيارة.
2. ستظهر سحابة نقاط حمراء (Particles) حول السيارة.
3. ابدأ تحريك السيارة ببطء باستخدام الكيبورد (W و A و D)؛ ستلاحظ تطابق المسار الأخضر مع حركة السيارة بدقة متناهية.

6. سجل المشاكل والحلول (Troubleshooting Log)

1. مشكلة (TF Fighting) وارتعاش السيارة:
 - السبب: كان كل من المحاكى وباكدج `rf2o` يحاولان التحكم في مكان السيارة في نفس الوقت.
 - الحل: تم ضبط `'publish_tf': False` في ملف إعدادات `rf2o` لمنعه من التدخل في شجرة التحويلات، والاكتفاء بإرسال البيانات عبر `Topic` فقط.
2. مشكلة طيران المسار خارج الخريطة (Axes Inversion):
 - السبب: اختلاف الإطار المرجعي بين الليدار في المحاكى وبين خوارزمية `rf2o`، مما أدى لتبديل وتدوير محاور الحركة (أصبح السير للأمام يسجل كحركة جانبية معكوسة).
 - الحل: تم التعديل في كود البايثون (`odomCB`) بتبديل قيم `X` و `Y` وضرب `Y` في سالب لعكس التأثير.
3. مشكلة تحذير `[laser] frame not found` في RViz:
 - السبب: كود الفلتر كان مبرمجاً لرسم ليدار وهمي على إطار اسمه `laser/`، وهو غير موجود في سيارتنا.
 - الحل: تم إزالة علامات `/` (Leading slashes) وتغيير اسم الإطار برمجياً ليصبح `roboracer_1` لي مطابق جسم السيارة.
4. تشتت النقاط عند الاصطدام بالحيطان:
 - السبب: ارتداد فيزيائي سريع من المحاكى واختفاء بيانات الليدار (Occlusion).
 - الحل: هذا سلوك طبيعي 100% للفلتر. الحل هو تجنب الاصطدام عبر خوارزميات القيادة، أو إعادة ضبط الـ `2D Pose Estimate` في حال حدوث حادثة.

7. Autonomous Control (المرحلة القادمة)

للتشغيل كوضع ذاتي القيادة (بعد التأكد من عمل الـ PF بشكل مثالي):

Bash

```
docker exec -it autodrive_roboracer_api bash
cd /home/autodrive_devkit
colcon build --packages-select wall_following
source install/setup.bash
ros2 run wall_following wall_follow
```

Note to Engineer: ROS2 Humble وبيئة ICRA هذا الملف يضمن توافق النظام مع نسخة 2026. Network Host تعمل على نفس الـ Containers تأكد دائماً من أن جميع الـ